The *static* keyword means that a member – like a field or method – belongs to the class itself, rather than to any specific instance of that class. **As a result, we can access static members without the need to create an instance of an object.**

We'll begin by discussing the differences between static and non-static fields and methods. Then, we'll cover static classes and code blocks, and explain why non-static components can't be accessed from a static context.

The *static* Fields (Or Class Variables)

In Java, when we declare a field *static*, exactly a single copy of that field is created and shared among all instances of that class.

It doesn't matter how many times we instantiate a class. There will always be only one copy of *static* field belonging to it. The value of this *static* field is shared across all objects of the same class. **From the memory perspective, static variables are stored in the heap memory.**

Imagine a class with several instance variables, where each new object created from this class has its own copy of these variables. However, if we want a variable to track the number of objects created we use a static variable instead. This allows the counter to be incremented with each new object:

```
public class Car {

   private String name;

   private String engine;

   public static int numberOfCars;

   public Car(String name, String engine) {

      this.name = name;

      this.engine = engine;

      numberOfCars++;

   }

   // getters and setters

}
```

As a result, the static variable *numberOfCars* will be incremented each time we instantiate the *Car* class. Let's create two *Car* objects and expect the counter to have a value of two:

```
@Test
public void whenNumberOfCarObjectsInitialized_thenStaticCounterIncreases() {
  new Car("Jaguar", "V8");
  new Car("Bugatti", "W16");
  assertEquals(2, Car.numberOfCars);
}
```

As we can see *static* fields can come in handy when:

- the value of the variable is independent of objects
- the value is supposed to be shared across all objects

Lastly, it's important to know that static fields can be accessed through an instance (e.g. *ford.numberOfCars*++) or directly from the class (e.g. *Car.numberOfCars*++). The latter is preferred, as it clearly indicates that it's a class variable rather than an instance variable.

The *static* Methods (Or Class Methods)

Similar to *static* fields, *static* methods also belong to a class instead of an object. So, we can invoke them without instantiating the class. Generally, we use *static* methods to perform an operation that's not dependent upon instance creation.

For example, we can use a *static* method to share code across all instances of that class:

```
static void setNumberOfCars(int numberOfCars) {
  Car.numberOfCars = numberOfCars;
}
```

Additionally, we can use *static* methods to create utility or helper classes. Some popular examples are the JDK's *Collections* or *Math* utility classes, Apache's *StringUtils,* and Spring Framework's *CollectionUtils*.
The same as for *static* fields, *static* methods can't be overridden. This is because *static* methods in Java are resolved at compile time, while method overriding is part of Runtime Polymorphism.

The following combinations of the instance, class methods, and variables are valid:

1. instance methods can directly access both instance methods and instance variables
2. instance methods can also access *static* variables and *static* methods directly
3. *static* methods can access all *static* variables and other *static* methods
4. *static* methods can't access instance variables and instance methods directly. They need some object reference to do so.

The *static* Code Blocks

Generally, we'll initialize *static* variables directly during declaration. **However, if the *static* variables require multi-statement logic during initialization we can use a *static* block instead.**

For instance, let's initialize a *List* object with some predefined values using *static* block of code:

```
public class StaticBlockDemo {

  public static List<String> ranks = new LinkedList<>();

  static {

    ranks.add("Lieutenant");

    ranks.add("Captain");

    ranks.add("Major");

  }
```

```
    static {
        ranks.add("Colonel");
        ranks.add("General");
    }
}
```

As we can see, it wouldn't be possible to initialize a *List* object with all the initial values along with the declaration. So, this is why we've utilized the *static* block here.

A class can have multiple *static* members. The JVM will resolve the *static* fields and *static* blocks in the order of their declaration. To summarize, the main reasons for using *static* blocks are:

- to initialize *static* variables needs some additional logic apart from the assignment
- to initialize *static* variables with a custom exception handling

The *static* Inner Classes

Java allows us to create a class within a class. It provides a way of grouping elements we use in a single place. This helps to keep our code more organized and readable.

As we can see, it wouldn't be possible to initialize a *List* object with all the initial values along with the declaration. So, this is why we've utilized the *static* block here.

A class can have multiple *static* members. The JVM will resolve the *static* fields and *static* blocks in the order of their declaration. To summarize, the main reasons for using *static* blocks are:

- to initialize *static* variables needs some additional logic apart from the assignment
- to initialize *static* variables with a custom exception handling

Java allows us to create a class within a class. It provides a way of grouping elements we use in a single place. This helps to keep our code more organized and readable.

```
public class Singleton  {

   private Singleton() {}

   private static class SingletonHolder {

      public static final Singleton instance = new Singleton();

   }

   public static Singleton getInstance() {

      return SingletonHolder.instance;

   }

}
```

We use this method because it doesn't require any synchronization and is easy to learn and implement.